

Implementation Plan (Execution Sequence)

Note to AI Agent: Execute these steps sequentially within the Antigravity workspace to avoid dependency conflicts.

Phase 1: Backend Infrastructure & Spatial Logic

1. Initialize Python FastAPI microservice architecture.
2. Provision PostgreSQL database and execute `CREATE EXTENSION postgis;`.
3. Build SQLAlchemy models mapping to the schema in Doc 05.
4. Implement the Agmarknet API ingestion route and the Smart Price XGBoost model endpoint.
5. Implement the Haversine spatial query for the `GET /api/v1/crops/nearby` endpoint to sort listings by distance:

$$d = 2R \arcsin \left(\sqrt{\sin^2 \left(\frac{\Delta\phi}{2} \right) + \cos(\phi_1) \cos(\phi_2) \sin^2 \left(\frac{\Delta\lambda}{2} \right)} \right)$$

Phase 2: Edge-AI & Offline Assets

1. Load the quantized MobileNetV2 model (INT8 `crop_disease_v1.tflite`).
2. Construct the local SQLite database for disease mappings and localized treatments.
3. Fetch PlantVillage, PlantDoc, and Indian Agriculture Crop Disease datasets via Kaggle API.
4. Structure directories for categorical `train` and `validation` splits.
5. Train the MobileNetV2 architecture locally utilizing CUDA/cuDNN.
6. Quantize the trained model to INT8 and export as `crop_disease_v1.tflite` directly into the Expo assets directory.

Phase 3: Upaj & Mandi Frontend Scaffolding

1. Initialize the React Native Expo Monorepo.
2. Configure NativeWind and apply the global design tokens specified in Doc 04.
3. Build the primary Tab Navigators for both Upaj and Mandi roles.

Phase 4: Hardware Integration & AI UI

1. Integrate `react-native-ble-plx` in Upaj's Weather module. Implement the Bluetooth scanning and hotspot credential transfer logic.
2. Integrate `react-native-fast-tflite` with `expo-camera`. Build the tensor preprocessing function to resize images to 224×224 pixels before inference.

Phase 5: Marketplace Handshake & Ledger Analytics

1. Wire the Upaj marketplace listing form to the FastAPI backend. Implement the $\pm 5\%$ constraint slider.

2. Wire the Mandi discover feed to ingest the PostGIS spatial results.
3. Implement `Linking.openURL('tel:...')` for P2P direct calling on both applications.
4. Implement the Farm Ledger logic to automatically parse API sales and calculate ROI:

$$\text{Crop ROI (\%)} = \left(\frac{\text{Crop Income} - \text{Crop Expenses}}{\text{Crop Expenses}} \right) \times 100$$